MLflow for GenAI

Tracing, evaluation and agent registration

Lecture 4 brings the course agent together: it retrieves arXiv research and writes a LinkedIn post about a requested topic. We record how it executes, assess whether its output meets the task, and package a version for deployment in the next lecture.

What you should be able to do

- Distinguish experiment tracking, the model registry, tracing and evaluation.
- Configure Unity Catalog trace storage and explain which identity needs access.
- Create parent and child spans, attach useful metadata, and investigate a failed request.
- Run the custom agent with retrieval, Genie and optional session memory.
- Build evaluation cases, choose scorers, and check the quality of an LLM judge.
- Log and register an agent while keeping evaluation and release decisions separate.

Reading map

Pages	Topic
2–5	MLflow components, storage, setup and permissions
6–9	Tracing APIs and the current agent implementation
10–14	Evaluation cases, scorers, execution and calibration
15–18	Logging, troubleshooting, homework and implementation map

Before starting: complete the Lecture 2 data pipeline and Lecture 3 retrieval, Genie and Lakebase setup. Use your own environment configuration and assigned resources. The repository pins MLflow 3.16.1; keep the course dependency set together.

These notes accompany the 4 November 2026 lecture. Work through notebooks 4.0–4.4 in order. The examples below identify where they assume an initialized configuration or agent. Deployment and production monitoring are covered later.

Place the MLflow components

Experiment tracking records runs, parameters, metrics and artifacts. The model registry manages named model versions. Tracing records the operations inside an execution. Evaluation applies explicit criteria to outputs and, when available, their execution evidence.

Component	Use in this course
Experiment tracking	Record evaluation metrics, code and configuration while logging the agent.
Model registry	Register the logged agent as a Unity Catalog model version for the Model Serving path.
MLflow tracing	Inspect requests, tool calls, memory access and nested execution. MLflow tracing was introduced in 2024 and is OpenTelemetry-compatible.
Evaluation	Measure task behavior with Python checks, LLM judges and human review.

Model flavors and interfaces

A flavor describes how MLflow logs and loads a model or application. Traditional ML has integrations for frameworks such as scikit-learn and PyTorch, plus custom `PythonModel` implementations. Agent frameworks also have integrations. The course uses custom Python orchestration through `mlflow.pyfunc.ResponsesAgent`.

The ResponsesAgent interface defines request, response and stream-event structures. It does not supply the research workflow or choose the tools for us. Those decisions live in `ArxivAgent`. Implementing a generator or inheriting ResponsesAgent does not, by itself, make network calls asynchronous.

Why log and register an agent?

For the Model Serving route demonstrated in the course, deployment consumes a registered model version. Databricks Apps is the other route we will explore next week. Evaluating an application before deployment is useful in both cases; an App does not need to use the same model-registry deployment workflow.

Langfuse, Phoenix, Logfire and Opik also provide tracing and evaluation capabilities. Shared tracing concepts and OpenTelemetry support do not make their storage, governance, integrations or evaluation features identical. We use MLflow here because it fits the Databricks course environment.

Decide where trace data belongs

Traditional ML runs commonly contain training parameters and metrics. GenAI traces often contain full user requests, retrieved passages, intermediate tool results and generated responses. That makes trace access part of the application's data-governance design.

Object	Course location and purpose
ML experiments and runs	Workspace experiments organize training or logging runs and their artifacts.
GenAI trace data	Unity Catalog tables in the configured catalog and schema; the experiment provides an interface for exploring them.
Registered models	Unity Catalog model versions, which can be shared across appropriately configured workspaces.

Workspace administration and data access

Workspace-managed trace access relies on workspace experiment permissions. A workspace administrator may therefore see sensitive trace contents. Unity Catalog storage lets the organization govern access through data privileges instead. A workspace-admin role alone is not a guarantee of access to every UC dataset.

Check effective permissions, not just role names: ownership, group membership, inherited grants, default workspace-catalog access and other administrator roles can change the result. Validate the runtime identity and human readers separately.

ML artifacts can also be sensitive. Training examples, model artifacts, paths and logged outputs may contain confidential information. The distinction is that GenAI tracing routinely captures conversational content, not that regular ML experiments are always harmless.

Control the recording as well as the reader

Decide which inputs and outputs are necessary for diagnosis. Remove secrets and avoid recording unnecessary personal data. Redaction or truncation must retain enough evidence for the checks you intend to run. A groundedness judge cannot verify a claim against passages that were never retained or supplied.

Unity Catalog also makes trace data available for SQL analysis. Separate environments and use cases deliberately; do not assume an experiment name creates an isolation boundary between users who can read the same underlying tables.

Read more Unity Catalog trace storage and governance

Set up Unity Catalog trace storage

Notebook 4.0 creates or selects the configured experiment and validates its trace location. The catalog and schema must already exist. The following is the creation pattern; use the notebook's existence and binding checks when rerunning it.

```
import os
import mlflow
from mlflow.entities.trace_location import UnityCatalog

# cfg is loaded for your environment from project_config.yml.
os.environ["MLFLOW_TRACING_SQL_WAREHOUSE_ID"] = cfg.warehouse_id
location = UnityCatalog(
    catalog_name=cfg.catalog,
    schema_name=cfg.schema,
    table_prefix="llmops",
)
experiment = mlflow.set_experiment(
    experiment_name=cfg.experiment_name,
    trace_location=location,
)
```

Finding an experiment with the right name is not enough. Notebook 4.0 checks the catalog, schema and prefix of its existing binding and stops if they differ. It does not silently convert an incompatible existing experiment.

What appears under Tables

Object	Type
`llmops_otel_annotations`	Table
`llmops_otel_logs`	Table
`llmops_otel_metrics`	Table
`llmops_otel_spans`	Table
`llmops_trace_metadata`	View
`llmops_trace_unified`	View

Inspect the views to understand the available trace fields before writing analysis queries. They provide a useful starting point for later monitoring. The annotations object supports trace annotations; it is not an additional application-memory store.

Platform availability: check regional and network support for your workspace, especially private-network ingestion. The lecture discussed Zerobus connectivity constraints; treat that discussion as a point-in-time deployment consideration, not a universal prohibition on production use.

Grant access and search traces

The identity running the agent must be able to write its traces. This may be your user during notebook exploration and a service principal (SPN) in deployment. A successful interactive test does not prove that the deployed identity has the same permissions.

Actor or operation	Access to check
Runtime trace writer	Explicit SELECT and MODIFY on the trace tables, SELECT on the views, and USE CATALOG / USE SCHEMA on parents.
Trace reader	SELECT on the intended trace tables/views and the required parent USE privileges.
SQL trace search	CAN USE on the selected SQL warehouse, in addition to the data permissions.

Notebook 4.0 reads the configured service principal application ID and grants access to objects with the `llmops_` prefix. Parent privileges must already be in place. For these trace tables, Databricks specifically requires explicit `SELECT` and `MODIFY`; do not rely only on `ALL PRIVILEGES`.

```
import os
import mlflow

# cfg is the environment configuration; the experiment exists.
os.environ["MLFLOW_TRACING_SQL_WAREHOUSE_ID"] = cfg.warehouse_id
experiment = mlflow.set_experiment(cfg.experiment_name)
traces = mlflow.search_traces(
    experiment_ids=[experiment.experiment_id],
    filter_string="tags.git_sha = 'local'",
    max_results=20,
)
display(traces)  # Databricks notebook display
```

Why the SQL warehouse matters

The course queries traces stored in UC through a SQL warehouse. Set its ID in the process that searches traces, not just in a different notebook or deployment step. The environment variable keeps the search call portable between trace-storage configurations.

If search returns nothing, first check the experiment, environment, filter value and time range. If it fails, inspect warehouse access and table privileges. Allow for trace ingestion delay before concluding that an execution was not recorded.

Read more Required trace privileges and warehouse configuration

Build a trace from nested spans

A trace represents an execution. Trace information describes its status, duration and metadata; trace data contains the spans that explain the steps. A span has its own identifier, timing, inputs, outputs and status, and can refer to a parent span.

```
import mlflow
from mlflow.entities import SpanType

@mlflow.trace(span_type=SpanType.TOOL)
def add_numbers(x: int, y: int) -> int:
    return x + y

@mlflow.trace(span_type=SpanType.CHAIN)
def process_query(x: int, y: int) -> int:
    total = add_numbers(x, y)
    return total * 2

process_query(2, 3)
```

process_query · parent CHAIN span
└── **add_numbers** · child TOOL span
One trace, two spans: the child runs while the parent is active.

Calling `add_numbers()` on its own creates a separate root execution. The parent relationship comes from the active tracing context, not the names of the functions or the fact that they share an experiment.

Use a manual span when you need control

```
with mlflow.start_span(name="calculation", span_type=SpanType.TOOL) as span:
    span.set_inputs({"x": 2, "y": 3})
    result = 2 + 3
    span.set_outputs({"result": result})
```

The decorator captures function inputs and outputs conveniently. A manual span lets you select what to record and precisely scope the measured operation. Start it before the work whose latency you want to measure.

Use meaningful types such as AGENT, CHAIN, LLM, TOOL, RETRIEVER, MEMORY, EMBEDDING and RERANKER. Types support interpretation and downstream evaluation. Choosing a type does not automatically collect every provider-specific field.

Attach context and investigate failures

A session groups a conversation; a request identifies one invocation. Several requests may share a session ID while producing separate traces. Code version and deployment metadata let you compare behavior across releases.

```
# Call inside an active traced function or span.
mlflow.update_current_trace(
    metadata={"mlflow.trace.session": session_id},
    tags={
        "git_sha": git_sha,
        "deployment_name": deployment_name,
        "model_version": model_version,
    },
    client_request_id=request_id,
)
```

The current agent reads release tags from `GIT_SHA`, `DATABRICKS_APP_NAME` or `MODEL_SERVING_ENDPOINT_NAME`, and `MODEL_VERSION`. Local fallbacks make experimentation possible; deployment must supply real values for release traceability.

Symptom	Evidence to inspect
Slow response	Which span dominates elapsed time? Look for slow retrieval, repeated calls and retries.
Unsupported claim	What passages were retrieved, and did the model actually receive them?
Wrong follow-up answer	Was the intended session loaded? Is history missing or duplicated?
Unexpected usage	How many LLM/tool steps ran, and was provider usage captured?
No answer	Did the loop hit its limit, did a tool fail, or was the response extraction wrong?

Tokens and cost are evidence to verify

A generic trace decorator does not guarantee that model names, token counts or cost will appear. Framework autologging and provider adapters may supply them. With custom instrumentation, inspect the returned usage fields and record the appropriate span attributes. Missing usage must not be interpreted as zero consumption.

For the current repository, `call_llm()` has no explicit LLM tracing decorator or manual span. The outer agent and tool spans do not alone provide a complete LLM-call breakdown. Add or verify dedicated LLM instrumentation when adapting the example.

A successful trace is not a quality verdict. The function may finish normally and still return an irrelevant answer, a fabricated citation or the iteration-limit message.

Read the arXiv agent implementation

The course application turns a request about research into a LinkedIn draft supported by actual papers. The data pipeline prepares the evidence before an agent request arrives.

Data preparation

arXiv PDFs and metadata → parsing and Delta tables

Paper chunks → Lakebase pgvector · Paper metadata → Genie

Request execution

User topic → load session → model and tool loop

→ research-grounded LinkedIn draft → save session

`src/arxiv_curator/agent.py` defines `ArxivAgent`, a ResponsesAgent subclass. It creates the model client, optional Lakebase memory, a local `search_papers` tool and managed Genie MCP tools. The system prompt is loaded through configuration from `prompts/linkedin_content.txt`.

Method	Responsibility
`predict_stream`	AGENT span; read request identifiers, load history and invoke the loop.
`call_and_run_tools`	CHAIN span; assemble messages, add trace metadata, run the loop and save new history.
`call_llm`	Use the Responses API and convert a returned function call or final text into an output item.
`handle_tool_call` / `execute_tool`	Parse arguments, execute the registered tool and append an output with the matching call ID.
`predict`	Collect completed output items into a ResponsesAgentResponse.

Understand the current boundaries

The loop is capped at ten iterations. In this implementation, a model call and a tool execution consume separate iterations; ten iterations does not mean ten complete model–tool cycles. The model adapter selects the first returned function call. Supporting multiple calls in one turn requires additional handling.

Although the public method yields response events, the current loop collects its events before yielding them. It is not token-by-token streaming. These details matter when interpreting latency or extending the implementation.

Invoke the agent and preserve sessions

Notebook 4.2 loads the environment configuration, obtains the Genie space ID, selects the experiment and constructs the agent. Call it in memory before packaging it for an endpoint.

```
from uuid import uuid4
from mlflow.types.responses import ResponsesAgentRequest

# agent is an initialized ArxivAgent from notebook 4.2.
session_id = str(uuid4())
request = ResponsesAgentRequest(
    input=[{
        "role": "user",
        "content": "Write a LinkedIn post about agent memory.",
    }],
    custom_inputs={
        "session_id": session_id,
        "request_id": str(uuid4()),
    },
)
response = agent.predict(request)

# Extract text rather than displaying the response envelope.
data = response.model_dump()
text = "\n".join(
    part["text"]
    for item in data["output"] if item.get("type") == "message"
    for part in item.get("content", [])
    if part.get("type") == "output_text"
)
print(text)
```

Test a follow-up deliberately

Send a new request such as “Make that shorter and keep the citations,” retaining `session_id` but generating a new `request_id`. With memory enabled, the agent loads previous messages and saves the new request and output items.

Choose how history enters the agent. If the same session is already loaded from Lakebase, also replaying all prior messages in the request can duplicate context. Without persistent memory, the caller must supply any conversation history needed for the task.

Inspect the result and its trace

- Check that the output is a post about the requested topic, not just a list of papers.
- Open the tool span and inspect the query and retrieved evidence.
- Confirm the trace has the expected session and request identifiers.
- Try a new session and verify that unrelated history does not carry over.

The current memory decorators label `memory_load` as RETRIEVER and `memory_save` as CHAIN. Read the actual names and operations; do not expect a MEMORY type merely because the operation uses memory.

Start evaluation with expected behavior

Observability explains how an output was produced. Evaluation asks whether it meets the task. An answer can be generated without errors and still be wrong. Conversely, a useful refusal may be the correct outcome when the evidence is insufficient.

Dimension	Traditional ML	Research-writing agent
Target	Compare predictions with defined outcomes.	Assess task-specific criteria; several posts may be acceptable.
Evidence	Labels and measurable prediction errors.	User request, retrieved papers, history and explicit expectations.
Methods	Metrics such as precision, recall or MAE.	Code checks, human review and model-based judges.

Define one case completely

Input: “Write a LinkedIn post about agent memory.”
Context: the available arXiv papers and any relevant prior turn.
Expected behavior: address memory, cite real retrieved papers, preserve their claims, follow the post format and avoid invented findings.

Build a small reviewed dataset

- Normal requests spanning the intended research topics.
- Follow-ups that require earlier context.
- Topics for which retrieval finds little or no supporting evidence.
- Out-of-scope requests and ambiguous instructions.
- Known failures, such as invented identifiers or iteration-limit responses.

Start with a manageable set, for example 10–20 carefully reviewed cases, then expand as you discover failures. This is an initial development set, not a statistically sufficient acceptance sample. Generating many similar prompts does not create meaningful coverage.

Why easy checks can miss the problem

A user says “Same issue as last month.” The assistant responds, “Can you describe the problem?” That answer can be short, polite and superficially relevant while ignoring accessible history. Add a context-use criterion and supply the history to the evaluator; a final-answer-only check cannot establish whether it was used correctly.

Review examples with the people who understand the task, describe the failure patterns, and only then choose the metrics. Keep a separate holdout set for judging changes.

Choose metrics and scorers

A metric defines what you want to measure; a scorer implements a way of measuring it. Different scorers require different evidence. Select them according to the failure you need to detect.

Approach	Useful for	Limitation
Python scorer	Length, structure, required fields or known tool events.	A passing format check says little about meaning.
Built-in LLM judge	A starting point for relevance, correctness or retrieval quality.	Needs the inputs, expectations or trace evidence required by that judge.
Guidelines	Task-specific natural-language rules, commonly pass/fail.	Vague criteria or missing context can produce misleading verdicts.
Custom judge	Explicit rubrics and numeric or categorical outcomes.	Must itself be tested and calibrated.
Human review	Ambiguous cases and business acceptance.	Reviewers need a shared rubric and can disagree.

A simple deterministic check

```
from mlflow.genai import scorer

# Example for a prediction function that returns plain text.
@scorer
def under_350_words(outputs: str) -> bool:
    return len(outputs.split()) < 350
```

The repository scorer `word_count_check` supports several output shapes, but primarily reads the first content item. Normalize the full final answer when you extend it. The condition is strictly fewer than 350 words, so 350 fails.

The prompt and scorer do not express the same target. The agent prompt asks for 150–250 words; the current check only rejects 350 or more. An empty answer can pass a length check. Add explicit completeness and task-success checks.

Notebook 4.4 runs word count, polite tone and hook-in-post checks. The package also defines scope and keyword-based paper-mention checks, but they are not all selected in that evaluation call. Finding the word “paper” is not evidence that a citation exists or supports a claim.

Read more Full MLflow scorer API reference

Write guidelines and custom judges

Guidelines provide a convenient way to apply a plain-language rule. Make each rule concrete and decide which inputs it needs. For a research-writing agent, tone and opening structure are separate from factual support.

```
from mlflow.genai.scorers import Guidelines

# cfg is loaded; this endpoint is available to the caller.
professional_tone = Guidelines(
    name="professional_tone",
    guidelines="The response must use a polite, professional tone.",
    model=f"databricks:/{cfg.llm_endpoint}",
)
```

Use a custom rubric when the outcome needs more detail

```
from typing import Literal
from mlflow.genai.judges import make_judge

support_judge = make_judge(
    name="research_support",
    instructions=(
        "Assess {{ outputs }} for the request in {{ inputs }} "
        "against the supplied paper excerpts in {{ expectations }}. "
        "Return supported if every substantive research claim is "
        "supported by those excerpts; unsupported if a claim "
        "contradicts or goes beyond them; insufficient_evidence "
        "if the necessary excerpts are missing."
    ),
    feedback_value_type=Literal[
        "supported", "unsupported", "insufficient_evidence"
    ],
    model=f"databricks:/{cfg.llm_endpoint}",
)
```

This is an example rubric to test, not a verified factuality checker. Supply the relevant paper excerpts in each row's `expectations`. Define how to handle uncertainty rather than forcing a confident verdict with missing evidence.

Built-in correctness judges can use reference expectations. Retrieval-groundedness judges need retrieval evidence. A custom judge can also use trace-based evaluation capabilities; choose that path when the execution, rather than just the final text, is the object of judgment.

An integer return type does not itself enforce a 1–5 rubric. Describe the levels precisely and inspect returned assessments. Changing the judge model, instructions or evidence can change its behavior and requires renewed evaluation.

Read more Creating and configuring custom MLflow judges

Run evaluation and inspect individual cases

MLflow can score existing input/output pairs, or call a prediction function for each input. Scoring fixed outputs isolates scorer behavior. Calling the agent tests the full application, including live model and tool dependencies.

```
import mlflow
from arxiv_curator.evaluation import (
    word_count_check, polite_tone_guideline, hook_in_post_guideline,
)

# agent and the experiment are initialized as in notebook 4.4.
def predict_fn(question: str):
    result = agent.predict({
        "input": [{"role": "user", "content": question}]
    })
    return result.output[-1].content

with open(eval_inputs_path) as f:
    eval_data = [
        {"inputs": {"question": line.strip()}}
        for line in f if line.strip()
    ]

results = mlflow.genai.evaluate(
    predict_fn=predict_fn,
    data=eval_data,
    scorers=[word_count_check, polite_tone_guideline,
             hook_in_post_guideline],
)
print(results.metrics)
```

The input key `question` must match the prediction function parameter. This course wrapper returns content blocks, not a plain string, despite the older notebook return annotation. Keep the output shape consistent with your scorers.

Use a repeatable review loop

- Read the output, each assessment and its rationale before relying on aggregate metrics.
- Inspect the trace for false passes and failures. Identify the responsible step.
- Record the agent configuration, prompt, data snapshot and judge configuration.
- Add a regression case for the failure, make one targeted change and rerun the comparison.

The provided `eval_inputs.txt` has ten short research requests and no reference expectations. It exercises the pipeline but does not fully test citations, groundedness, history or the LinkedIn writing contract. “Latest paper” also depends on the data snapshot; use explicit dates or a frozen dataset when comparing versions.

> The lecture demonstration included iteration-limit outputs. Treat these as incomplete tasks, even if a length or tone check passes. Notebook 4.4 runs evaluation before registration but does not enforce an acceptance threshold automatically.

Calibrate the judge against human review

An LLM judge is another model in the system. Its verdict can be wrong because the rubric is ambiguous, relevant context is missing, the model is inconsistent or the task itself has several defensible interpretations.

Align on evidence and criteria first

- Have multiple qualified reviewers assess the same examples with the same written rubric and context.
- Inspect human–human disagreement before treating one reviewer as ground truth.
- Compare judge decisions and rationales with the reviewed cases.
- Separate examples used to improve the judge from examples used to test it.
- Repeat validation when the judge, rubric or application domain changes.

MLflow supports judge alignment using human assessments, including a SIMBA-based optimizer. Alignment can improve agreement on the provided data, but it does not prove that the resulting judge generalizes. Notebook 4.3 describes the workflow; its alignment section is instructional rather than a completed optimization with real human annotations.

Avoid calibrating only to one person

A judge can learn one reviewer's preferences, blind spots or inconsistent labeling. The useful question is: **is the judge good enough to replace a human for this particular review task?** Define the task, allowed error and cost of mistakes before deciding.

The Alternative Annotator Test

Alt-Test compares the LLM and a held-out human against the same remaining human annotators, repeats that comparison across annotators, and accounts for cost–benefit assumptions and multiple testing. Its winning-rate criterion of at least 0.5 applies within the paper's procedure and chosen settings; it is not a generic accuracy threshold.

Use the paper's method and assumptions if you want a statistical replacement claim. A small calibration exercise can reveal rubric problems without being sufficient to justify replacing human review.

Keep operational decisions explicit

Do not assume a cheaper model is adequate, or that a different provider automatically eliminates bias. Compare candidates on the reviewed task. Escalate uncertain or high-impact cases according to the application's requirements.

Read more MLflow judge alignment

Read more Calderon, Reichart and Dror · ACL 2025 · Alt-Test

Log and register an agent version

Logging creates a model artifact with code and configuration. Registration creates a version under a named Unity Catalog model. Updating an alias points a readable name at a chosen version. None of these actions deploys an endpoint.

Package the current entry point

```
# src/arxiv_curator/agent_model.py
from mlflow.models import ModelConfig, set_model
from arxiv_curator.agent import ArxivAgent

set_model(ArxivAgent(**ModelConfig().to_dict()))
```

Notebook 4.4 supplies the configuration at logging time, including the model endpoint, prompt, catalog, schema, Genie space, embedding endpoint and Lakebase settings. It explicitly includes the installed package through `code_paths`.

```
# After evaluation, using notebook 4.4's initialized variables.
with mlflow.start_run(tags={"git_sha": git_sha, "run_id": run_id}):
    model_info = mlflow.pyfunc.log_model(
        name="agent",
        python_model=str(package_path / "agent_model.py"),
        code_paths=[str(package_path)],
        input_example=test_request,
        model_config=model_config,
    )
    mlflow.log_metrics(results.metrics)

model_name = f"{cfg.catalog}.{cfg.schema}.arxiv_agent"
version = mlflow.register_model(
    model_uri=model_info.model_uri,
    name=model_name,
    env_pack="databricks_model_serving",
)
mlflow.MlflowClient().set_registered_model_alias(
    name=model_name, alias="latest-model", version=version.version
)
```

The course uses serverless environment packing. Run logging and registration in the compatible environment and verify the resulting dependency package. Replace illustrative release identifiers such as `abc` and `unset` before relying on them for provenance.

Registration is not authorization. The current logging call does not declare automatic resource-based authentication. Deployment must configure the runtime identity and its access to the gateway, Genie, Lakebase and trace storage. The alias is mutable; record the concrete version used for a release.

Read more Logging and registering agents for Model Serving

Questions and troubleshooting

Question or symptom	What to check
No traces returned	Confirm experiment ID, environment, filter values and ingestion delay. A session ID must actually have been attached to the trace.
SQL search fails	Check MLFLOW_TRACING_SQL_WAREHOUSE_ID, warehouse CAN USE and UC table/view permissions for this identity.
Experiment exists but setup fails	Read its trace_location. The same name can refer to an experiment created with a different storage binding.
Writer works locally but fails when deployed	Test the service principal, not the notebook user. Confirm explicit trace grants and parent USE privileges.
Tool icon is red	Read span status and error details. Icon colors can indicate a span type rather than failure.
Tokens or model fields are missing	Check provider usage and tracing integration. The course custom call needs explicit instrumentation for a full LLM breakdown.
Repeated or missing context	Check session IDs and whether history is both loaded from Lakebase and replayed in the request.
Max iterations reached	Read the tool/model sequence. Count loop iterations using the actual implementation; improve termination or routing before raising the cap.
Scores look good but the answer is wrong	Check what was evaluated. Tone, length and keyword matches do not establish factual support.

When should evaluation run?

Before deployment: compare changes on reviewed examples and regression cases. This is the focus of Lecture 4. **Inside a request:** a check may block, redirect or escalate execution, adding latency. **After deployment:** sampled production traces can reveal drift and new failure patterns; this is covered in a later lecture.

An offline evaluation result is not automatically a runtime guardrail. To change behavior during a request, application code must consume the result and implement the next action.

What if the judge lacks context?

Include an explicit insufficient-evidence outcome in a custom rubric where needed. A generated score is not evidence that the judge received everything required to assess the task.

Homework

Apply the lecture workflow to your own use case. Build on the agent and tools from Lecture 3. The aim is a version you can explain, trace and evaluate before deploying it next week.

1. Assemble and trace the agent

- Create an `agent.py` with a clear request entry point and a bounded model/tool loop.
- Add meaningful spans for the agent, tools and model calls. Record only the evidence you need.
- Propagate session and request IDs, plus code and deployment metadata.
- Show one successful trace and one failure or edge case. Explain which step caused the outcome.

2. Build and run evaluation

- Write a small, diverse set of cases with expected behavior and supporting context. Include at least a follow-up, missing-evidence case and known failure.
- Implement a useful deterministic check and a task-specific guideline or custom judge.
- Inspect individual outputs and rationales. Identify at least one case that a superficial check could incorrectly pass.
- Review the judge against human decisions; document ambiguity or disagreement. Use a holdout for assessing changes.

3. Prepare a version for deployment

- Log the agent's code, configuration and evaluation metrics. Register a model version if following the Model Serving exercise.
- Explain which identity will execute the agent and what resources it must access.
- Record the concrete version, relevant configuration and remaining limitations. Registration alone is not a release gate.

What to include in your pull request

Include the implementation, evaluation cases, scorer definitions and a concise explanation of the results. Link or attach redacted evidence from traces and evaluation. State what changed after inspection, what still fails and what you will test next. You may combine unfinished work from earlier lectures with this submission.

Before running the course test job: notebook 4.0 grants permissions; notebook 4.4 evaluates, registers a version and updates `latest-model`. Run against the intended development environment and prepared data. The job is not scheduled automatically.

Use `resources/testing_jobs/lecture4_tracing_evaluation.yml` for the sequential notebook workflow. Resolve environment and resource configuration before treating a failed infrastructure call as an agent-quality failure.

Implementation map and further reading

Repository file	Purpose
`notebooks/4.0.uc_traces_setup.py`	Experiment binding and trace-object grants.
`notebooks/4.1.tracing_implementation.py`	Decorators, spans, metadata and searches.
`notebooks/4.2.custom_agent.py`	Initialize and invoke the agent; inspect traces.
`notebooks/4.3.evaluation_theory.py`	Scorer examples, guidelines, custom judges and alignment concepts.
`notebooks/4.4.mlflow_log_register.py`	Run evaluation, log, register and set an alias.
`src/arxiv_curator/agent.py`	Agent orchestration and tracing.
`src/arxiv_curator/evaluation.py`	Scorers and evaluation helpers.
`src/arxiv_curator/agent_model.py`	Models-from-code entry point.
`prompts/linkedin_content.txt`	Research-writing instructions loaded by configuration.
`eval_inputs.txt`	Starting inputs; extend with reviewed expectations.
`project_config.yml`	Environment-specific resources and prompt path.

Documentation

- Unity Catalog trace setup and permissions
- Full MLflow scorer reference
- Custom judge construction and alignment with human feedback
- Logging and registering agents
- The Alternative Annotator Test · ACL 2025

Recommended book

Evals for AI Engineers by Shreya Shankar and Hamel Husain: *Systematically Measuring and Improving AI Applications*. Continue with its treatment of evaluation datasets, error analysis and evaluation workflows.

Use the course dependency versions for the exercises. Documentation may describe newer APIs or deployment paths; check version requirements before copying an example into the pinned environment.